

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 – SORTING



NAMA: Laluna Afril Diasyifa

NIM: 109082500009

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

1. Pengurutan (Sorting)

Pengurutan (*sorting*) adalah proses menata ulang sekumpulan data sehingga tersusun dalam urutan tertentu berdasarkan suatu nilai acuan (*key*). Tujuan pengurutan adalah memudahkan pencarian, pengolahan, maupun analisis data.

Secara umum, terdapat dua jenis urutan yang sering digunakan:

- Ascending (menaik), yaitu data disusun dari nilai terkecil ke nilai terbesar.
- Descending (menurun), yaitu data disusun dari nilai terbesar ke nilai terkecil.

Dalam pemrograman, terdapat berbagai algoritma yang dapat digunakan untuk melakukan pengurutan. Dua metode yang cukup sederhana dan sering dipelajari adalah Selection Sort dan Insertion Sort. Kedua algoritma tersebut memiliki kompleksitas waktu sebesar $O(n^2)$ pada kasus rata-rata maupun kasus terburuk, dengan n menyatakan jumlah data yang diurutkan.

2. Selection Sort (Pengurutan secara Seleksi)

Selection Sort merupakan metode pengurutan yang bekerja dengan cara memilih elemen bernilai paling kecil atau paling besar dari bagian data yang belum terurut, kemudian menempatkannya pada posisi yang sesuai.

Pada setiap tahap, algoritma ini melakukan dua aktivitas utama:

1. Mencari nilai ekstrem, yaitu nilai terkecil (untuk ascending) atau terbesar (untuk descending) pada bagian data yang belum terurut.
2. Menukar posisi elemen, yaitu memindahkan nilai yang ditemukan ke posisi yang seharusnya dengan melakukan pertukaran (*swap*).

Langkah-langkah Selection Sort (Ascending)

1. Tentukan nilai terkecil pada bagian data yang belum terurut.
2. Tukarkan nilai tersebut dengan elemen pertama pada bagian yang belum terurut.
3. Anggap posisi yang sudah ditempati nilai terkecil sebagai bagian yang telah terurut.
4. Ulangi proses pencarian dan pertukaran pada sisa data hingga seluruh elemen tersusun dengan benar.

Metode ini terus memperluas bagian data yang telah terurut dari kiri ke kanan sampai seluruh data selesai diurutkan.

2.3 Insertion Sort (Pengurutan secara Penyisipan)

Insertion Sort adalah algoritma pengurutan yang menyusun data dengan cara memasukkan setiap elemen ke posisi yang tepat pada bagian data yang sudah terurut sebelumnya.

Berbeda dengan Selection Sort yang mencari nilai ekstrem pada seluruh sisa data, Insertion Sort berfokus pada satu elemen yang sedang diproses, kemudian menentukan posisi yang sesuai di antara elemen-elemen yang telah terurut.

Proses yang terjadi pada algoritma ini meliputi:

1. Menentukan posisi yang tepat untuk suatu elemen.
2. Menggeser elemen-elemen lain apabila diperlukan.
3. Menyisipkan elemen tersebut pada posisi yang telah ditemukan.

Langkah-langkah Insertion Sort (Descending)

1. Anggap elemen pertama sudah terurut.
2. Ambil elemen berikutnya sebagai data yang akan ditempatkan.
3. Bandingkan elemen tersebut dengan data yang sudah terurut di sebelah kirinya.
4. Geser elemen yang nilainya lebih kecil ke kanan untuk memberi ruang.
5. Sisipkan elemen yang sedang diproses ke posisi yang sesuai.
6. Ulangi langkah tersebut hingga seluruh data berhasil diurutkan.

Dengan cara kerja tersebut, bagian data yang telah terurut akan terus bertambah setiap kali iterasi berlangsung sampai semua elemen tersusun sesuai urutan yang diinginkan.

B. Guided

1. Sorting

```
package main

import "fmt"

func SelectionSortArray(arr *[8]int) {
    var idx_min, i, j int

    for i = 0; i < len(arr) - 1; i++ { //perulangan luar
        idx_min = i;
        for j = i + 1; j < len(arr); j++ { //perulangan dalam
            if arr[j] < arr[idx_min] { //kondisional
                idx_min = j
            }
        }
        if idx_min != i { //swap
            arr[i], arr[idx_min] = arr[idx_min], arr[i]
        }
    }
}

func main() {
    var arrAngka [8]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka indeks ke-%d: ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
    fmt.Println()

    SelectionSortArray(&arrAngka)
    fmt.Println("=== SETELAH SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}
```

Output :

```
PS D:\luna\109082500009_Laluna-Afril-Diasyifa> go run "d:\luna\109082500009_Laluna-Afril-Diasyifa
Week12-Modul13\Guided\Guided-1\guided1.go"
Masukkan data angka indeks ke-0: 1
Masukkan data angka indeks ke-1: 3
Masukkan data angka indeks ke-2: 4
Masukkan data angka indeks ke-3: 5
Masukkan data angka indeks ke-4: 6
Masukkan data angka indeks ke-5: 2
Masukkan data angka indeks ke-6: 4
Masukkan data angka indeks ke-7: 6

=== SEBELUM SORTING ===
1 3 4 5 6 2 4 6
=== SETELAH SORTING ===
1 2 3 4 4 5 6 6
```

2. Sorting

```
package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS) - 1; i++ { //loop luar
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ { //loop dalam
            mencari nilai ekstrim
            if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting
                terkecil ke terbesar
            }
        }
    }
}
```

```

        idx_min = j
    }
}
if idx_min != i {
    sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
}
}
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa ke-%d ---\n",
i)
        fmt.Print("Nama: ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM: ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK: ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama: %s\n", structMHS[i].nama)
        fmt.Printf("NIM: %s\n", structMHS[i].nim)
        fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)
    fmt.Println("=== SETELAH SORTING ===")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama: %s\n", structMHS[i].nama)
        fmt.Printf("NIM: %s\n", structMHS[i].nim)
        fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}

```

Output :

```

PS D:\luna\109082500009_Laluna-Afril-Diasyi
week12-Modul13\Guided\Guided-2\guided2.go"
--- Masukkan Data Mahasiswa ke-0 ---
Nama: a
NIM: 1
IPK: 3.44
--- Masukkan Data Mahasiswa ke-1 ---
Nama: b
NIM: 2
IPK: 3.55
--- Masukkan Data Mahasiswa ke-2 ---
Nama: c
NIM: 3
IPK: 3.66
--- Masukkan Data Mahasiswa ke-3 ---
Nama: d
NIM: 4
IPK: 3.77
--- Masukkan Data Mahasiswa ke-4 ---
Nama: e
NIM: 5
IPK: 3.45
=== SEBELUM SORTING ===
--- Data Mahasiswa ke-0 ---
Nama: a
NIM: 1
IPK: 3.44
--- Data Mahasiswa ke-1 ---
Nama: b
NIM: 2
IPK: 3.55
--- Data Mahasiswa ke-2 ---
Nama: c
NIM: 3
IPK: 3.66
--- Data Mahasiswa ke-3 ---
Nama: d
NIM: 4
IPK: 3.77
--- Data Mahasiswa ke-4 ---
Nama: e
NIM: 5
IPK: 3.45
=====
=== SETELAH SORTING ===
--- Data Mahasiswa ke-0 ---
Nama: a
NIM: 1
IPK: 3.44
--- Data Mahasiswa ke-1 ---
Nama: e
NIM: 5
IPK: 3.45
--- Data Mahasiswa ke-2 ---
Nama: b
NIM: 2
IPK: 3.55
--- Data Mahasiswa ke-3 ---
Nama: c
NIM: 3
IPK: 3.66
--- Data Mahasiswa ke-4 ---
Nama: d
NIM: 4
IPK: 3.77
=====

```

3. Sorting

```
package main

import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    for i = 1; i <= n - 1; i++ {
        temp = T[i]
        j = i
        for j > 0 && temp > T[j - 1] {
            T[j] = T[j - 1]
            j--
        }
        T[j] = temp
    }
}

func main() {
    var data arrInt
    var n, i int

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Masukkan data: ")

    for i = 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}
```

```

    fmt.Println("\nData sebelum sorting: ")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)

    fmt.Println("\n\nData setelah sorting (Descending): ")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    fmt.Println()
}

```

Output :

```

PS D:\luna\109082500009_Laluna-Afril-Diasyifa> go run "d:\luna\109082500009_Laluna-Afril-Diasyifa\
Week12-Modul13\Guided\Guided-3\guided3.go"
Masukkan jumlah data: 3
Masukkan data:
16 2 7

Data sebelum sorting:
16 2 7

Data setelah sorting (Descending):
16 7 2

```

4. Sorting

```

package main

import "fmt"

type mahasiswa struct {
    nama string
    nim string
    ipk float64
}

```

```

}

type arrMhs [100]mahasiswa

func insertionSortStruct(T *arrMhs, n int) {
    var temp mahasiswa
    var i, j int

    for i = 1; i < n; i++ {
        temp = T[i]
        j = i - 1
        for j >= 0 && T[j].nama < temp.nama {
            T[j + 1] = T[j]
            j--
        }
        T[j + 1] = temp
    }
}

func main() {
    var data arrMhs
    var n, i int

    fmt.Print("Masukkan jumlah mahasiswa: ")
    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Println("\nData mahasiswa ke-", i + 1)

        fmt.Print("Nama: ")
        fmt.Scan(&data[i].nama)

        fmt.Print("NIM: ")
        fmt.Scan(&data[i].nim)

        fmt.Print("IPK: ")
        fmt.Scan(&data[i].ipk)
    }

    fmt.Println("\nData sebelum sorting: ")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
    }

    insertionSortStruct(&data, n)

    fmt.Println("\nData setelah sorting (Descending
berdasarkan Nama): ")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
    }
}

```

```
}
```

Output :

```
PS D:\luna\109082500009_Laluna-Afril-Diasyifa> go run "d:\luna\109082500009_Laluna-Afril-Diasyifa\
Week12-Modul13\Guided\Guided-4\guided4.go"

Data mahasiswa ke- 1
Nama: a
NIM: 1
IPK: 3.44

Data mahasiswa ke- 2
Nama: b
NIM: 2
IPK: 3.11

Data mahasiswa ke- 3
Nama: c
NIM: 3
IPK: 3.56

Data sebelum sorting:
a 1 3.44
b 2 3.11
c 3 3.56

Data setelah sorting (Descending berdasarkan Nama):
c 3 3.56
b 2 3.11
a 1 3.44
```

C. Unguided

1. Rumah Kerabat

- 1) Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu.

Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 2 7 9 13 15 27 39 75 133 189 1 4 9

Keterangan: Terdapat 3 daerah dalam contoh input, dan di masing-masing daerah mempunyai 5, 6, dan 3 kerabat.

Jawaban :

```
package main

import "fmt"

func selectionSort(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMin := i
        for j := i+1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }
        temp := arr[idxMin]
        arr[idxMin] = arr[i]
```

```

        arr[i] = temp
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        var rumah []int
        for j := 0; j < m; j++ {
            var nomor int
            fmt.Scan(&nomor)
            rumah = append(rumah, nomor)
        }

        selectionSort(rumah, m)

        for j := 0; j < m; j++ {
            if j > 0 {
                fmt.Print(" ")
            }
            fmt.Print(rumah[j])
        }
        fmt.Println()
    }
}

```

Output :

```

PS D:\luna\109082500009_Laluna-Afril-Diasyifa> go run "d:\luna\109082500009_Laluna-Afril-Diasyifa\
Week12-Modul13\Unguided\Unguided-1\uguided1.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9

```

Penjelasan :

Program tersebut digunakan untuk mengurutkan nomor rumah pada beberapa kelompok data secara menaik (*ascending*) menggunakan algoritma Selection Sort. Program diawali dengan membaca nilai *n* yang menyatakan jumlah kelompok data yang akan diproses. Selanjutnya, untuk setiap kelompok, program membaca nilai *m* sebagai banyaknya nomor rumah yang akan dimasukkan. Setiap nomor rumah yang diinput oleh pengguna disimpan ke dalam sebuah *slice* bernama *rumah*. Setelah seluruh data pada kelompok tersebut berhasil dikumpulkan, fungsi `selectionSort` dipanggil untuk mengurutkan data. Algoritma ini bekerja dengan cara mencari nilai terkecil pada bagian data yang belum terurut, kemudian menukarnya dengan elemen pada posisi yang seharusnya. Proses tersebut dilakukan berulang hingga semua nomor rumah tersusun dari yang terkecil hingga yang terbesar. Setelah pengurutan selesai, program menampilkan hasilnya dalam satu baris dengan setiap nomor dipisahkan oleh spasi. Langkah yang sama akan terus diulang sampai seluruh kelompok data selesai diproses.

2. Kerabat Dekat

- 2) Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program **kerabat dekat** yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 13 12 8 2 15 27 39 75 133 189 8 4 2

Keterangan: Terdapat 3 daerah dalam contoh masukan. Baris kedua berisi campuran bilangan ganjil dan genap. Baris berikutnya hanya berisi bilangan ganjil, dan baris terakhir hanya berisi bilangan genap.

Petunjuk:

- Waktu pembacaan data, bilangan ganjil dan genap dipisahkan ke dalam dua array yang berbeda, untuk kemudian masing-masing diurutkan tersendiri.
- Atau, tetap disimpan dalam satu array, diurutkan secara keseluruhan. Tetapi pada waktu pencetakan, mulai dengan mencetak semua nilai ganjil lebih dulu, kemudian setelah selesai cetaklah semua nilai genapnya.

Jawaban :

```
package main

import "fmt"

func selectionSortAsc(arr []int, n int) {
```

```

        for i := 0; i < n-1; i++ {
            idxMin := i
            for j := i+1; j < n; j++ {
                if arr[j] < arr[idxMin] {
                    idxMin = j
                }
            }
            temp := arr[idxMin]
            arr[idxMin] = arr[i]
            arr[i] = temp
        }
    }

func selectionSortDesc(arr []int, n int) {
    for i := 0; i < n-1; i++ {
        idxMax := i
        for j := i+1; j < n; j++ {
            if arr[j] > arr[idxMax] {
                idxMax = j
            }
        }
        temp := arr[idxMax]
        arr[idxMax] = arr[i]
        arr[i] = temp
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        var ganjil []int
        var genap []int

        for j := 0; j < m; j++ {
            var nomor int
            fmt.Scan(&nomor)
            if nomor%2 != 0 {
                ganjil = append(ganjil, nomor)
            } else {
                genap = append(genap, nomor)
            }
        }

        selectionSortAsc(ganjil, len(ganjil))
        selectionSortDesc(genap, len(genap))
    }
}

```

```

    first := true
    for j := 0; j < len(ganjil); j++ {
        if !first {
            fmt.Print(" ")
        }
        fmt.Print(ganjil[j])
        first = false
    }

    for j := 0; j < len(genap); j++ {
        if !first {
            fmt.Print(" ")
        }
        fmt.Print(genap[j])
        first = false
    }
    fmt.Println()
}

```

Output :

```

PS D:\luna\109082500009_Laluna-Afril-Diasyifa> go run "d:\luna\109082500009_Laluna-Afril-Diasyifa\week12-Modul13\Unguided\Unguided-2\tempCodeRunnerFile.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4

```

Penjelasan :

Program tersebut berfungsi untuk memisahkan nomor rumah menjadi dua kelompok, yaitu bilangan ganjil dan bilangan genap, kemudian mengurutkannya dengan aturan yang berbeda. Pada awal proses, program membaca nilai *n* yang menunjukkan jumlah kelompok data yang akan diproses. Untuk setiap kelompok, program membaca jumlah nomor rumah yang akan dimasukkan, lalu menerima setiap nomor rumah satu per satu. Selama proses input, program langsung memeriksa apakah suatu nomor termasuk bilangan ganjil atau genap dan menyimpannya ke dalam *slice* yang sesuai. Setelah seluruh data dalam kelompok tersebut berhasil dipisahkan, bilangan ganjil diurutkan secara menaik (*ascending*) menggunakan fungsi `selectionSortAsc`, sedangkan bilangan genap diurutkan secara menurun (*descending*) menggunakan fungsi `selectionSortDesc`. Setelah proses pengurutan selesai, program menampilkan seluruh bilangan ganjil yang telah terurut terlebih dahulu, kemudian dilanjutkan dengan seluruh bilangan genap yang juga sudah terurut. Proses ini dilakukan berulang untuk setiap kelompok data hingga seluruh data yang diberikan selesai diproses.

3. Jarak Data

- 1) Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	31 13 25 43 1 7 19 37 -5	1 7 13 19 25 31 37 43 Data berjarak 6
2	4 40 14 8 26 1 38 2 32 -31	1 2 4 8 14 26 32 38 40 Data berjarak tidak tetap

Jawaban :

```
package main  
  
import "fmt"
```

```

func insertionSort(arr []int, n int) {
    for i := 1; i < n; i++ {
        temp := arr[i]
        j := i
        for j > 0 && temp < arr[j-1] {
            arr[j] = arr[j-1]
            j = j - 1
        }
        arr[j] = temp
    }
}

func main() {
    var arr []int
    var input int

    for {
        fmt.Scan(&input)
        if input < 0 {
            break
        }
        arr = append(arr, input)
    }

    n := len(arr)
    if n > 0 {
        insertionSort(arr, n)
    }

    for i := 0; i < n; i++ {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(arr[i])
    }
    fmt.Println()

    if n < 2 {
        fmt.Println("Data berjarak tidak tetap")
    } else {
        jarak := arr[1] - arr[0]
        tetap := true
        for i := 1; i < n-1; i++ {
            if arr[i+1]-arr[i] != jarak {
                tetap = false
                break
            }
        }
    }
}

```

```

    }

    if tetap {
        fmt.Printf("Data berjarak %d\n", jarak)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}
}

```

Output :

```

PS D:\luna\109082500009_Laluna-Afril-Diasyifa> go run "d:\luna\109082500009_Laluna-Afril-Diasyifa\
Week12-Modul13\Unguided\Unguided-3\unguided3.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap

```

Penjelasan :

Program tersebut digunakan untuk mengurutkan sekumpulan bilangan sekaligus memeriksa apakah data yang dimasukkan memiliki jarak atau selisih yang tetap antar elemennya. Pada awalnya, program menerima input angka secara berulang dan menyimpannya ke dalam sebuah *slice*. Proses pembacaan data akan berhenti ketika pengguna memasukkan sebuah bilangan negatif yang berfungsi sebagai penanda akhir input. Setelah seluruh data terkumpul, program memanggil fungsi **insertionSort** untuk mengurutkan angka-angka tersebut dari nilai terkecil ke nilai terbesar. Metode ini bekerja dengan mengambil setiap elemen secara bertahap, kemudian menyisipkannya pada posisi yang tepat di antara data yang sudah terurut sebelumnya. Setelah proses pengurutan selesai, program menampilkan seluruh angka yang telah tersusun secara berurutan. Selanjutnya, program menghitung selisih antara dua elemen pertama dan menggunakan nilai tersebut sebagai acuan untuk memeriksa selisih antar elemen berikutnya. Jika seluruh selisih memiliki nilai yang sama, program akan menampilkan informasi bahwa data memiliki jarak tetap beserta besar jaraknya. Namun, apabila ditemukan perbedaan selisih pada salah satu pasangan elemen, program akan menyimpulkan bahwa data memiliki jarak yang tidak tetap dan menampilkan pesan yang sesuai.

4. Insertion Sort

- 2) Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
  id, judul, penulis, penerbit : string
  eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Lengkapi subprogram-subprogram dibawah ini, sesuai dengan I.S. dan F.S yang diberikan.

```
procedure DaftarkanBuku(in/out pustaka : DaftarBuku, n : integer)
{I.S. sejumlah n data buku telah siap para piranti masukan
 F.S. n berisi sebuah nilai, dan pustaka berisi sejumlah n data buku}

procedure CetakTerfavorit(in pustaka : DaftarBuku, in n : integer)
{I.S. array pustaka berisi n buah data buku dan belum terurut
 F.S. Tampilan data buku (judul, penulis, penerbit, tahun)
 terfavorit, yaitu memiliki rating tertinggi}

procedure UrutBuku( in/out pustaka : DaftarBuku, n : integer )
{I.S. Array pustaka berisi n data buku
 F.S. Array pustaka terurut menurun/mengecil terhadap rating.
 Catatan: Gunakan metoda Insertion sort}

procedure CetakSTerbaru( in pustaka : DaftarBuku, n integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan 5 judul buku dengan rating tertinggi
 Catatan: Isi pustaka mungkin saja kurang dari 5}

procedure CariBuku(in pustaka : DaftarBuku, n : integer, r : integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan salah satu buku (judul, penulis, penerbit, tahun,
 eksemplar, rating) dengan rating yang diberikan. Jika tidak ada buku
 dengan rating yang ditanyakan, cukup tuliskan "Tidak ada buku dengan
 rating seperti itu". Catatan: Gunakan pencarian biner/belah dua.}
```

Jawaban :

```
package main

import "fmt"

const nMax int = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n *int) {
    fmt.Scan(n)

    for i := 0; i < *n; i++ {
        fmt.Scan(
            &pustaka[i].id,
            &pustaka[i].judul,
            &pustaka[i].penulis,
```

```

        &pustaka[i].penerbit,
        &pustaka[i].eksemplar,
        &pustaka[i].tahun,
        &pustaka[i].rating,
    )
}
}

func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        return
    }

    maxRating := pustaka[0].rating
    idx := 0

    for i := 1; i < n; i++ {
        if pustaka[i].rating > maxRating {
            maxRating = pustaka[i].rating
            idx = i
        }
    }

    fmt.Printf(
        "%s %s %s %d\n",
        pustaka[idx].judul,
        pustaka[idx].penulis,
        pustaka[idx].penerbit,
        pustaka[idx].tahun,
    )
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    for i := 1; i < n; i++ {
        temp := pustaka[i]
        j := i

        for j > 0 && temp.rating > pustaka[j-1].rating {
            pustaka[j] = pustaka[j-1]
            j--
        }

        pustaka[j] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    limit := 5

    if n < 5 {

```

```

        limit = n
    }

    for i := 0; i < limit; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka DaftarBuku, n, r int) {
    kiri := 0
    kanan := n - 1
    found := false
    idx := -1

    for kiri <= kanan && !found {
        tengah := (kiri + kanan) / 2

        if pustaka[tengah].rating == r {
            found = true
            idx = tengah
        } else if r > pustaka[tengah].rating {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }

    if found {
        fmt.Printf(
            "%s %s %s %d %d %d\n",
            pustaka[idx].judul,
            pustaka[idx].penulis,
            pustaka[idx].penerbit,
            pustaka[idx].tahun,
            pustaka[idx].eksemplar,
            pustaka[idx].rating,
        )
    } else {
        fmt.Println("Tidak ada buku dengan rating seperti itu")
    }
}

func main() {
    var pustaka DaftarBuku
    var n, r int

    DaftarkanBuku(&pustaka, &n)

    CetakTerfavorit(pustaka, n)
}

```

```
UrutBuku(&pustaka, n)
```

```
Cetak5Terbaru(pustaka, n)
```

```
fmt.Scan(&r)
```

```
CariBuku(pustaka, n, r)
```

```
}
```


Output :

```
PS D:\luna\109082500009_Laluna-Afril-Diasyifa> go run "d:\luna\109082500009_Laluna-Afril-Diasyifa\week12-Modu113\Unguided\Unguided-4\unguided4.go"
6
B01 Algoritma_Dasar Budi Informatika_Press 10 2020 85
B02 Pemrograman_Go Andi Telkom_Press 5 2021 95
B03 Struktur_Data Citra IT_Books 15 2019 70
B04 Basis_Data Doni SQL_Pub 8 2022 88
B05 Jaringan_Komp Eka Net_Press 12 2018 75
B06 AI_Lanjut Fajar Future_Books 20 2023 92
Pemrograman_Go Andi Telkom_Press 2021
Pemrograman_Go
AI_Lanjut
Basis_Data
Algoritma_Dasar
Jaringan_Komp
88
Basis_Data Doni SQL_Pub 2022 8 88
```

Penjelasan :

Program tersebut merupakan aplikasi sederhana untuk mengelola data perpustakaan yang menyimpan informasi buku beserta nilai ratingnya. Pada awal proses, program membaca jumlah buku yang akan dimasukkan, kemudian menerima dan menyimpan seluruh data buku, seperti ID, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating ke dalam sebuah array. Setelah semua data berhasil disimpan, program menjalankan fungsi CetakTerfavorit untuk mencari buku dengan rating tertinggi dan menampilkan informasi penting dari buku tersebut. Selanjutnya, seluruh data buku diurutkan berdasarkan rating dari nilai tertinggi ke nilai terendah menggunakan algoritma Insertion Sort melalui fungsi UrutBuku. Setelah proses pengurutan selesai, fungsi Cetak5Terbaru digunakan untuk menampilkan lima judul buku yang berada pada posisi teratas dalam daftar hasil pengurutan. Pada tahap berikutnya, program menerima sebuah nilai rating sebagai kata kunci pencarian. Fungsi CariBuku kemudian melakukan pencarian menggunakan metode Binary Search pada data yang sudah terurut sehingga proses pencarian dapat dilakukan dengan lebih cepat dan efisien. Jika ditemukan buku dengan rating yang sesuai, program akan menampilkan informasi lengkap buku tersebut. Namun, apabila tidak ada buku yang memiliki rating yang dicari, program akan memberikan pesan bahwa buku dengan rating tersebut tidak ditemukan.

D. Kesimpulan

Berdasarkan pembelajaran pada modul ini, dapat disimpulkan bahwa algoritma Selection Sort dan Insertion Sort merupakan metode pengurutan yang penting untuk mengatur data sehingga lebih terstruktur dan mudah diolah. Kedua algoritma tersebut dapat digunakan dalam berbagai kasus, seperti mengurutkan data berdasarkan kriteria tertentu, memisahkan dan menyusun kelompok data, maupun menyiapkan data agar dapat dicari dengan lebih cepat menggunakan Binary Search. Selain itu, penggunaan struktur data seperti slice dan struct membantu dalam menyimpan serta mengelola data yang memiliki karakteristik berbeda dengan lebih terorganisir. Melalui latihan dan penerapan pada berbagai permasalahan, dapat dipahami bahwa kemampuan dalam mengelola proses perbandingan, penggeseran, pertukaran, dan pencarian data merupakan keterampilan dasar yang sangat penting dalam pemrograman. Dengan memahami konsep-konsep tersebut, pengembang dapat membuat program yang lebih efektif, terstruktur, dan mampu menyelesaikan permasalahan pengolahan data dengan baik.